

Python Assignment 2

Name: Harikarthikeyan V

Screenshots

Data Structures - List, Dictionary, Set& Conditional Statements

1. List Creation:

```
4]: # 1. LIST CREATION

#1.a create age list
age_list = [24, 25, 26, 27, 28]
print(age_list)

#1.b create name list
name_list = ["Arun", "Bala", "Charan", "Divya", "Esha"]
print(name_list)
```

[24, 25, 26, 27, 28]
['Arun', 'Bala', 'Charan', 'Divya', 'Esha']

2. List Operations / Modifications:

```
jupyter Data Structures - List, Dictionary, Set & Conditional Statement Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

[5]: #2. LIST OPERATIONS/MODIFICATIONS
# 2.a append "YAZHINI"
name_list.append("Yazhini")
print(name_list)

['Arun', 'Bala', 'Charan', 'Divya', 'Esha', 'Yazhini']

[6]: # 2.b INSERT 30 AT INDEX 2
age_list.insert(2, 30)
print(age_list)

[24, 25, 30, 26, 27, 28]

[7]: #2.c Remove the string "Yazhini" from name_list
name_list.remove("Yazhini")
print(name_list)

['Arun', 'Bala', 'Charan', 'Divya', 'Esha']

[8]: # 2.d Pop the last element from age_list
age_list.pop()
print(age_list)

[24, 25, 30, 26, 27]

[9]: # 2.e Extend the age_list with additional ages [29, 30, 26].
age_list.extend([29, 30, 26])
print(age_list)

[24, 25, 30, 26, 27, 29, 30, 26]

[10]: # 2.f Sort age_list in descending order.
age_list.sort(reverse=True)
print(age_list)

[30, 30, 29, 27, 26, 26, 25, 24]

[11]: # 2.g Find Max age, Min age and sum of all ages from age_list
```

```
1]: # 2.g Find Max age, Min age and sum of all ages from age_list.
max_age = max(age_list)
min_age = min(age_list)
total_age = sum(age_list)
print("Maximum age:", max_age)
print("Minimum age:", min_age)
print("Sum of all ages:", total_age)

Maximum age: 30
Minimum age: 24
Sum of all ages: 217
```

3. Accessing List Elements:

```
: # 3. Accessing List Elements
# 3.a Print the first element of name_list.
print(name_list[0])

Arun

: # 3.b Print the last element of name_list.
print(name_list[-1])

Esha

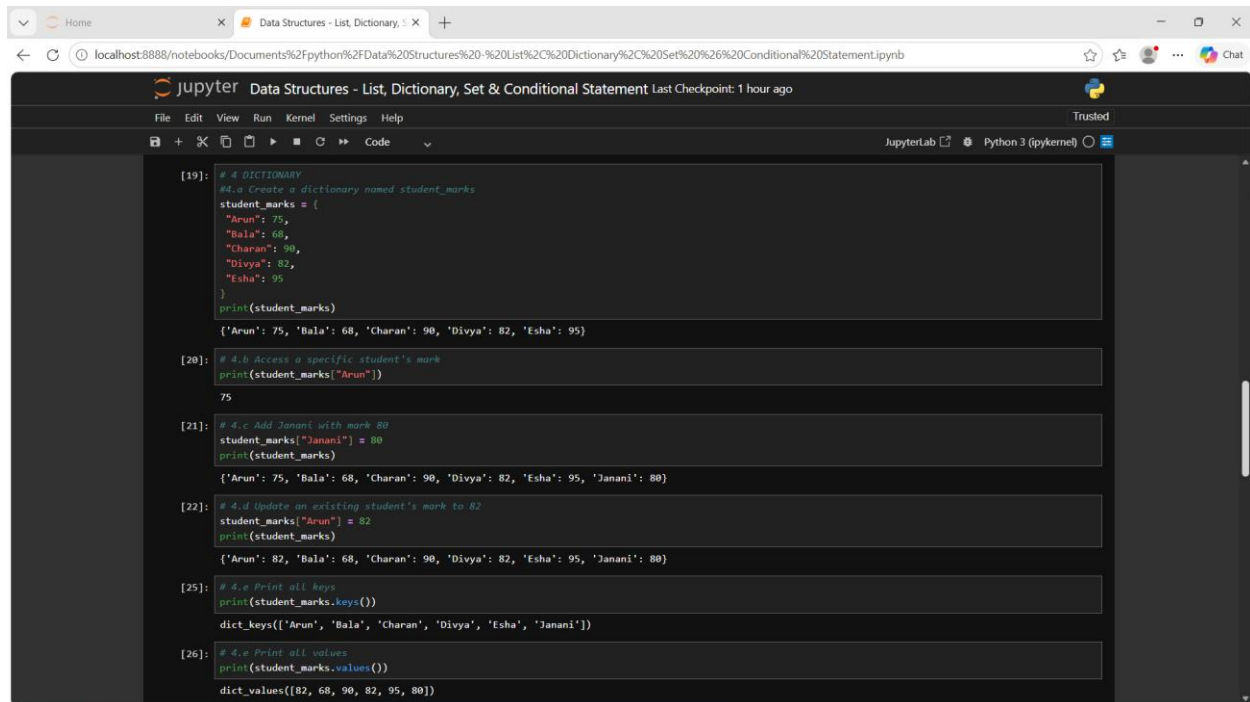
: # 3.c Print the elements from index 2 to index 4 in name_list.
print(name_list[2:5])

['Charan', 'Divya', 'Esha']

: # 3.d Print the elements of name_list in reverse order.
print(name_list[::-1])

['Esha', 'Divya', 'Charan', 'Bala', 'Arun']
```

4. Dictionary (Creation, Modification and Access):



The screenshot shows a JupyterLab notebook titled "Data Structures - List, Dictionary, Set & Conditional Statement". The notebook contains several code cells demonstrating dictionary operations:

```
[19]: # 4.1 DICTIONARY
#4.1 Create a dictionary named student_marks
student_marks = {
    "Arun": 75,
    "Bala": 68,
    "Charan": 90,
    "Divya": 82,
    "Esha": 95
}
print(student_marks)

{'Arun': 75, 'Bala': 68, 'Charan': 90, 'Divya': 82, 'Esha': 95}

[20]: # 4.2 Access a specific student's mark
print(student_marks["Arun"])

75

[21]: # 4.3 Add Janani with mark 80
student_marks["Janani"] = 80
print(student_marks)

{'Arun': 75, 'Bala': 68, 'Charan': 90, 'Divya': 82, 'Esha': 95, 'Janani': 80}

[22]: # 4.4 Update an existing student's mark to 82
student_marks["Arun"] = 82
print(student_marks)

{'Arun': 82, 'Bala': 68, 'Charan': 90, 'Divya': 82, 'Esha': 95, 'Janani': 80}

[25]: # 4.5 Print all keys
print(student_marks.keys())
dict_keys(['Arun', 'Bala', 'Charan', 'Divya', 'Esha', 'Janani'])

[26]: # 4.6 Print all values
print(student_marks.values())
dict_values([82, 68, 90, 82, 95, 80])
```

```
[ ]: # 4.e Print all keys
print(student_marks.keys())

dict_keys(['Arun', 'Bala', 'Charan', 'Divya', 'Esha', 'Janani'])

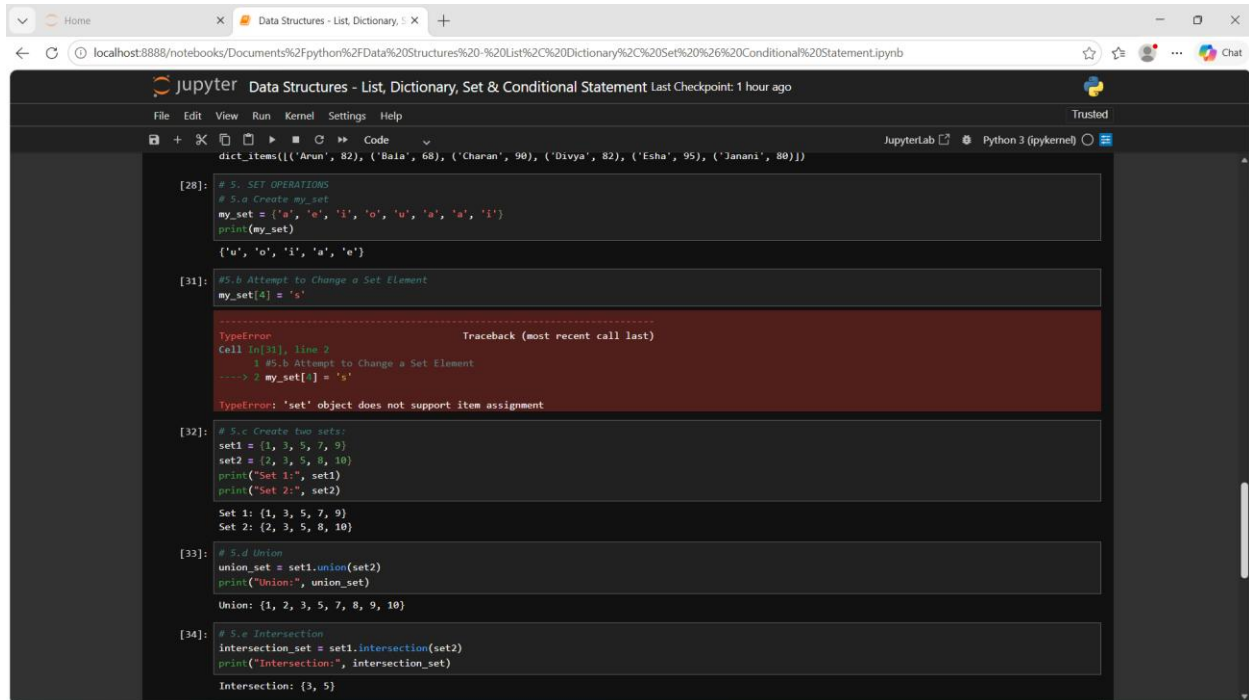
[ ]: # 4.e Print all values
print(student_marks.values())

dict_values([82, 68, 90, 82, 95, 80])

[ ]: #4.e Print key-value pairs
print(student_marks.items())

dict_items([('Arun', 82), ('Bala', 68), ('Charan', 90), ('Divya', 82), ('Esha', 95), ('Janani', 80)])
```

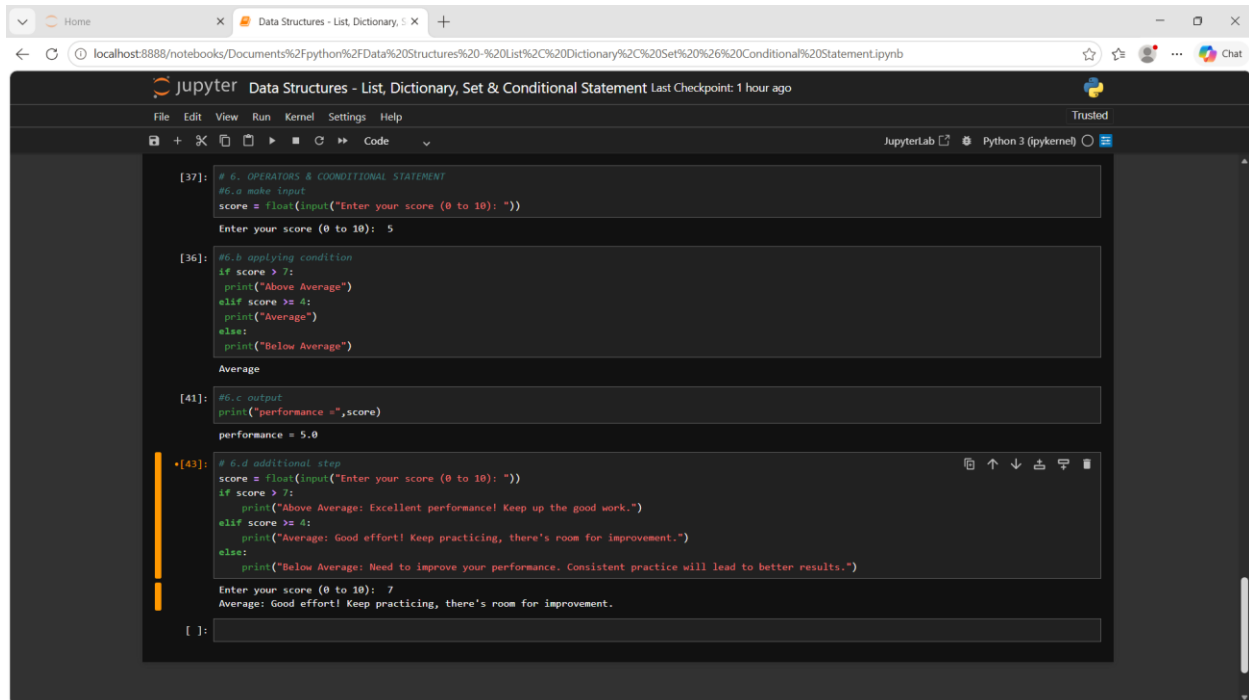
5. Sets (Operations):



The screenshot shows a Jupyter Notebook interface with the title "Data Structures - List, Dictionary, Set & Conditional Statement". The notebook contains several code cells demonstrating set operations:

- Cell [28]: Creates a set `my_set` from a list of characters and prints it. Output: `{'u', 'o', 'i', 'a', 'e'}`.
- Cell [31]: Attempts to change an element in the set `my_set[4] = 's'`. This results in a `TypeError: 'set' object does not support item assignment`.
- Cell [32]: Creates two sets, `set1` and `set2`, and prints them. Output: `Set 1: {1, 3, 5, 7, 9}` and `Set 2: {2, 3, 5, 8, 10}`.
- Cell [33]: Computes the union of `set1` and `set2` using `set1.union(set2)` and prints the result. Output: `Union: {1, 2, 3, 5, 7, 8, 9, 10}`.
- Cell [34]: Computes the intersection of `set1` and `set2` using `set1.intersection(set2)` and prints the result. Output: `Intersection: {3, 5}`.

6. Operators & Conditional Statements :



The screenshot shows a Jupyter Notebook interface with the title "Data Structures - List, Dictionary, Set & Conditional Statement". The notebook contains several code cells demonstrating conditional statements and operators:

- Cell [37]: Prompts the user to enter a score (0 to 10). Input: `5`.
- Cell [36]: Applies a condition to the score using an `if-elif-else` statement. Output: `Average`.
- Cell [41]: Prints the performance score. Output: `performance = 5.0`.
- Cell [43]: Prompts the user to enter a score (0 to 10). Input: `7`. The code then prints a message based on the score: `Average: Good effort! Keep practicing, there's room for improvement.`